Machine Learning

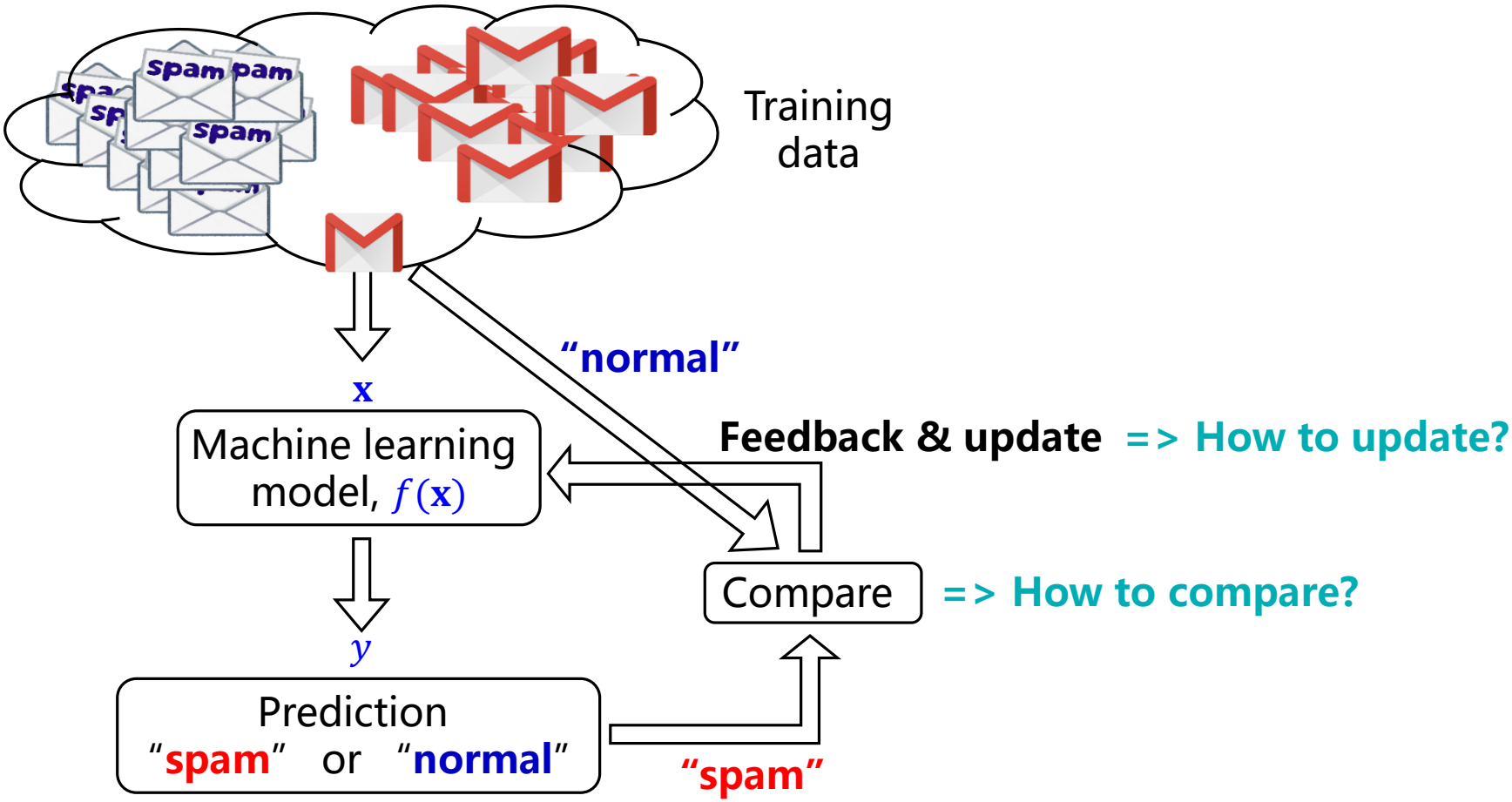
Machine learning background

Dong-Kyu Chae

PI of the Data Intelligence Lab @HYU
Department of Computer Science
Hanyang University

Data
Intelligence
LAB

Model Training Overview



Preliminaries

- **Scalar**
  - A scalar is a number, like 0, 1, 100, -0.368, etc
  - In general, **a function** is a scalar

Scalar Vector Matrix

$$1 \qquad \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

- **Vector**
  - A vector is a list of numbers (can be in a row or column)
  - In general, **one data example** can be represented by a vector
- **Matrix**
  - A matrix is an array of numbers (or, a list of vectors)
  - In general, the **entire dataset** can be represented by a matrix

Preliminaries

Scalar, vector, matrix in machine learning

- A model prediction, $f(\mathbf{x};\Theta)$ is a scalar
- Each data $\mathbf{x}$ is a vector, and the corresponding label is a scalar
  - mail#1 is represented as [256, 0, 3, 7, ….], and its label is '1'
- The entire (input) dataset is a matrix

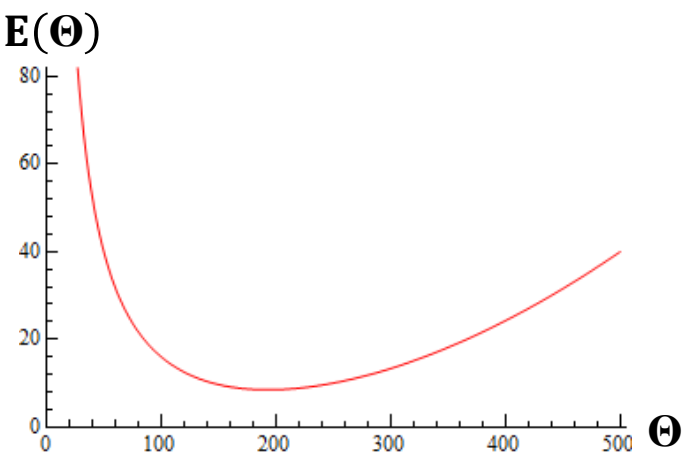
	# of words	# of attached files	# of links	# of malicious words	…..	spam
mail #1	256	0	3	7	…..	1 (Yes)
mail #2	56	1	0	3	…..	0 (No)
mail #3	24	1	0	1	…..	0
mail #4	672	0	0	0	…..	0
mail #5	67	2	4	3	…..	1
mail #6	48	0	2	6	…..	0
mail #7	79	1	3	8	…..	1
			…..			

Goal of Training

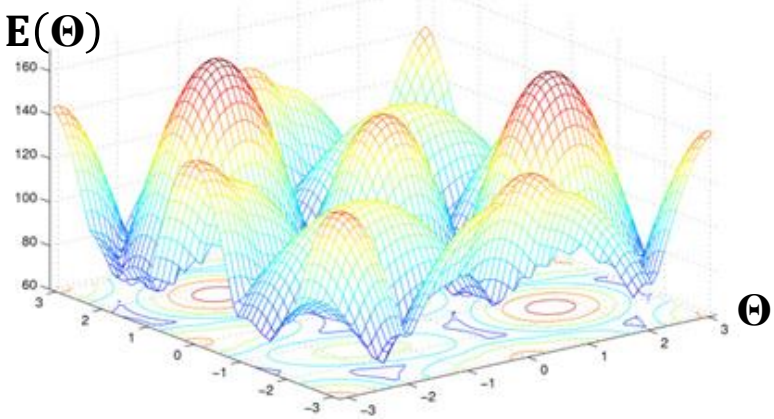
- **Training data:** $(\mathbf{x}_i, y_i) = (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots (\mathbf{x}_n, y_n)$
- **Model:** $f(\mathbf{x}; \boldsymbol{\Theta})$
- **Model parameters:** $\boldsymbol{\Theta}$
- **Goal: finding the optimal** $\boldsymbol{\Theta}$

Process

- **Step 1:** Define a cost (or, loss, error) function $\mathrm{E}(\boldsymbol{\Theta})$
  - Difference between the **model's prediction** and **actual value**
  - Example: $\mathrm{E}(\boldsymbol{\Theta}) = \sum_{i=1}^{n}(y_i - f(\mathbf{x}_i;\Theta))^2$
- The cost function $\mathrm{E}(\boldsymbol{\Theta})$ is also a scalar
- The forms of the cost function can be categorized by:



<convex function>



<non-convex function>

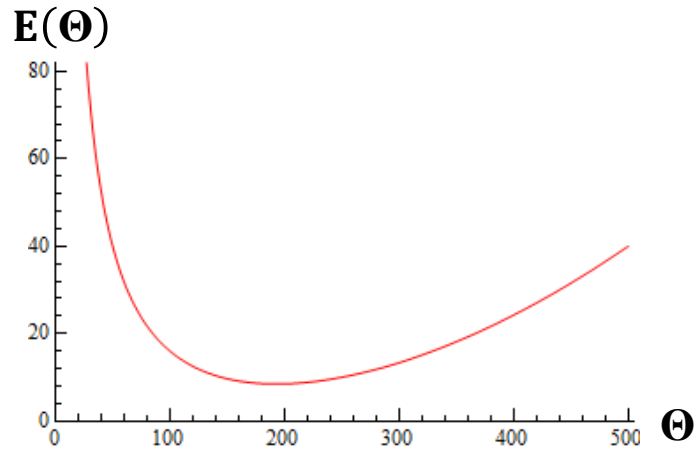
Process

- **Step 2:** Solve the **optimization problem**
  - Getting optimal $\boldsymbol{\Theta}^*$ that minimizes $\mathbf{E}(\boldsymbol{\Theta})$
  - $\boldsymbol{\Theta}^* = \underset{\boldsymbol{\Theta}}{\operatorname{argmin}}\, \mathrm{E}(\Theta)$
    - "argmin" means arguments of the minima, which are the points of a cost function where the function output is minimized.

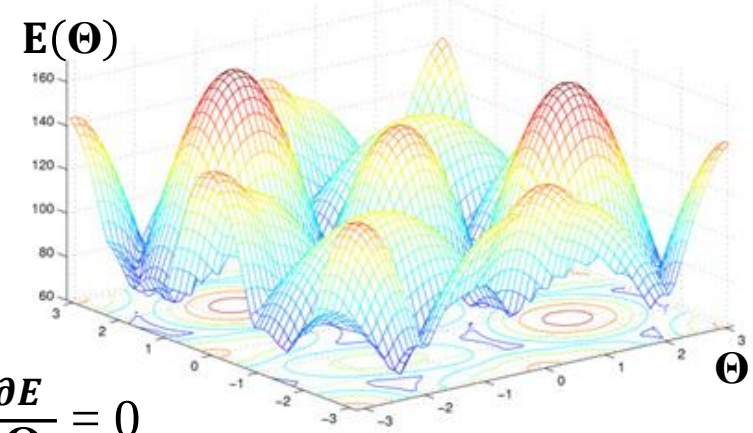
- **Various optimization can be applied depending on the complexity of the loss function**

Optimization

- **How to solve the optimization problem?**
  - **Case 1:** If the cost function **E** is **simple**,
    - You can directly find the optimal parameters via computing the point where the **derivative** of E ($\mathbf{E}'(\boldsymbol{\Theta})$) becomes zero => $\frac{\partial E}{\partial \boldsymbol{\Theta}} = 0$



  - **Case 2:** If the cost function **E** is **complex**,
    - Multiple model parameters…
    - Model parameters are aligned with each other…
    - We cannot directly solve the problem via $\frac{\partial E}{\partial \boldsymbol{\Theta}} = 0$



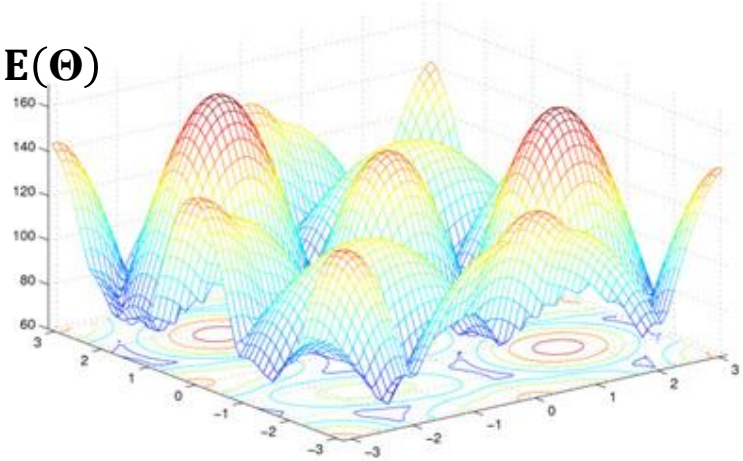
Optimization

- **A simple case** $\frac{\partial E}{\partial \Theta} = 0$
  - We need to compute the derivative of a scalar by a vector
- **Let a scalar** y **and a vector** $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$
  - The derivative of y by x is:

$$\frac{\partial y}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \frac{\partial y}{\partial x_2} & \cdots & \frac{\partial y}{\partial x_n} \end{bmatrix}$$

Optimization

- **A complex case:**
  - There are multiple minimum points
    - Local minima
    - Global minima
  - We cannot distinguish either it is global minima or local minima

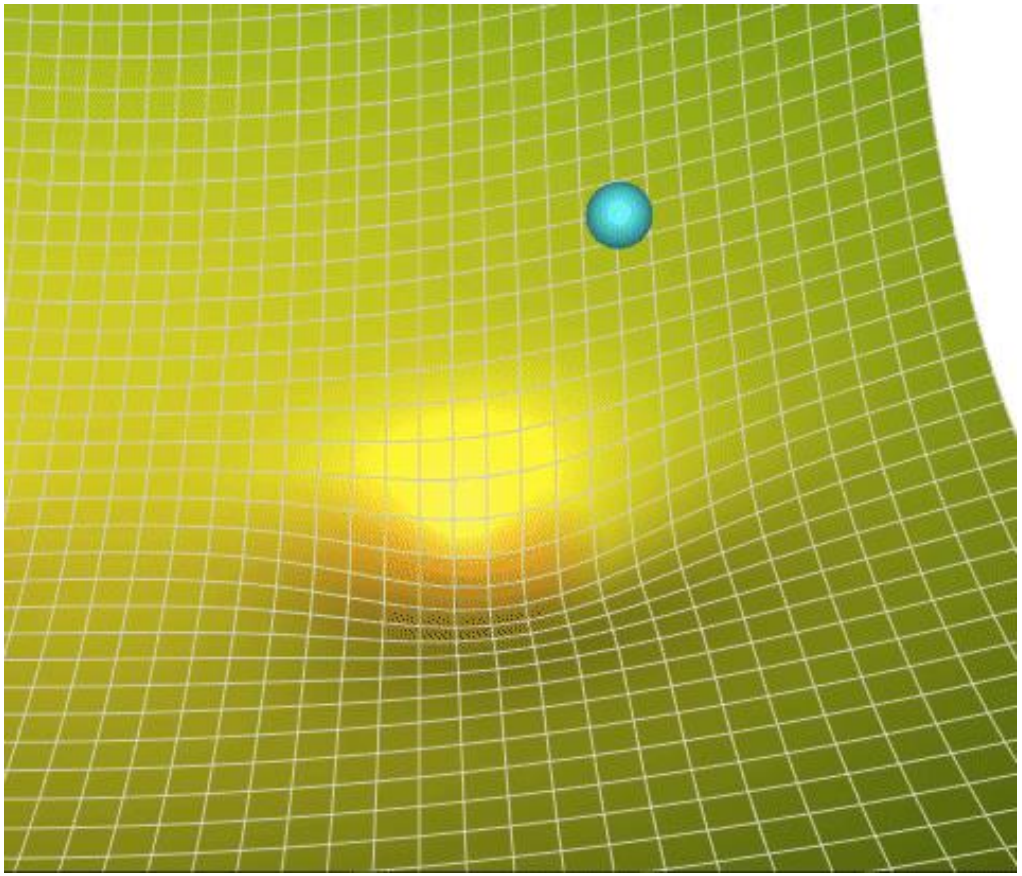


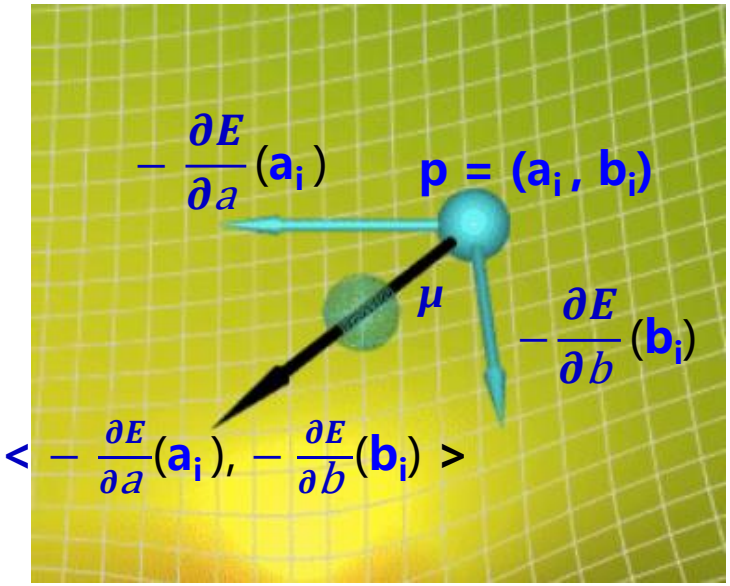
- **Solution: Gradient descent**
  - https://en.wikipedia.org/wiki/Gradient
  - The **gradient** of a function f at a point p is the **vector** whose elements are the **partial derivatives** of f at p

$$\nabla f(p) = \left[\frac{\partial f}{\partial x_1}(p) \quad \cdots \quad \frac{\partial f}{\partial x_n}(p) \right]$$

  - The gradient indicates the "direction in which the function f **increases** most quickly"

Gradient Descent Example

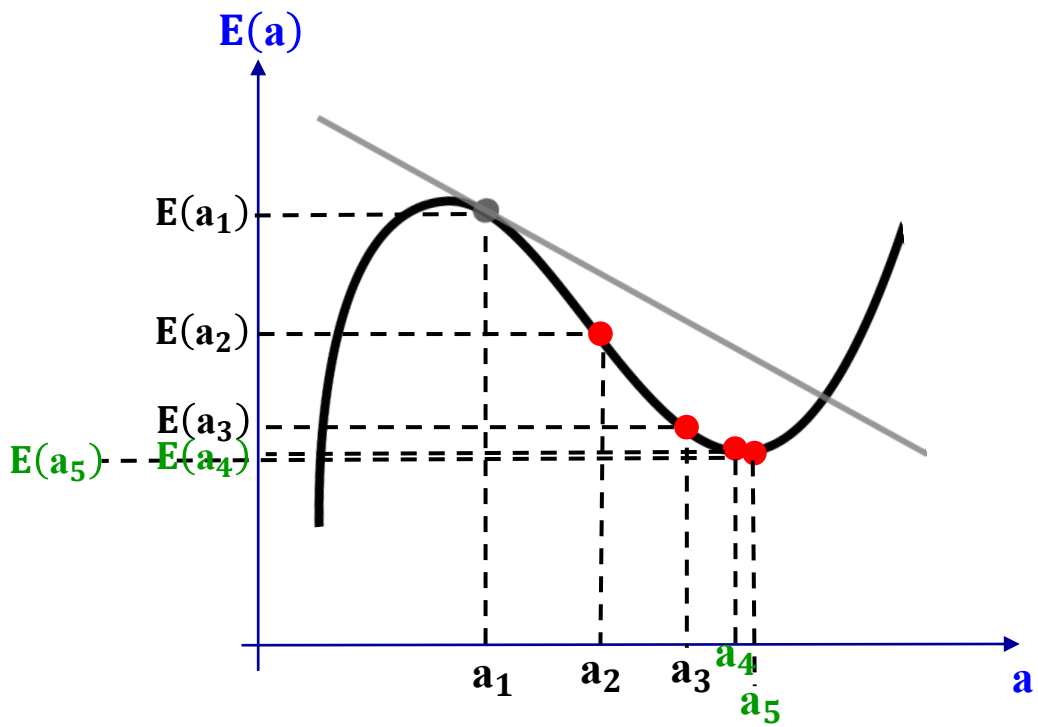




Gradient Descent

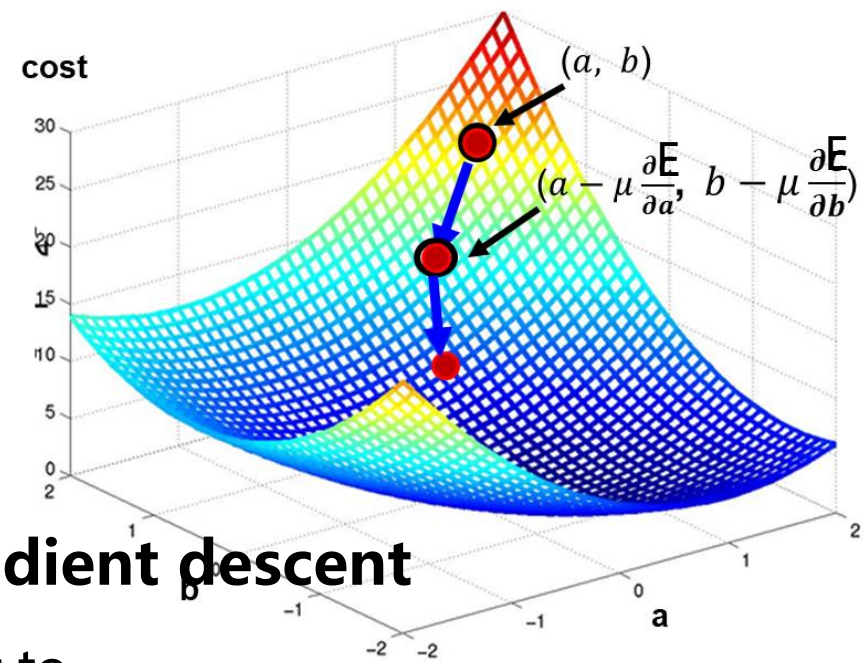
When to stop gradient descent

- Until the cost function **E** converges
  - Let δ be a convergence threshold, then we decide the convergence of E by:
  - If $\mathbf{E}(\mathbf{a}_{n-1}) - \mathbf{E}(\mathbf{a}_n) < \delta$
  - δ can be very small number like 0.001



Summary

- **Model Training?**
  - Define a cost function
  - Find optimal parameter values



- **Convex optimization:** $\frac{\partial E}{\partial \Theta} = 0$
- **Non-convex optimization: gradient descent**
  1. Randomly choose a starting point to travel the high-dimensional space
  2. Repeat (1) computing gradient and (2) moving the opposite direction of the gradient with a small step size
  3. Stop until the cost function converges (i.e., it reaches to a **local** minima)

Thank You